Software Engineering & Project Management (SEPM)

Energy & Utility Management System (EUMS)

Electricity · Water · Gas — billing, metering and customer service

Software Requirements Specification

Software process model: Waterfall · Combined SEPM + DBMS course project

Group number	Group __
Course and section	Software Engineering & Project Management (SEPM), Section __
Supervisor / instructor	Prof. _____
Institute	_____ Institute of Technology
Date	October 2026

#	Name	Registration number
1	Sushrut Lad	_____
2	Member 2	_____
3	Member 3	_____
4	Member 4	_____

Document	Software Requirements Specification (IEEE 830 structure)
Version	1.0 (baselined at end of Requirements phase; status columns updated at mid-semester)
Product	Energy & Utility Management System (EUMS)

1. Introduction

1.1 Purpose

This document specifies the requirements of the Energy & Utility Management System (EUMS). It is the agreed baseline between the development team and the course instructor, and it is the input to the design, implementation and testing phases of our Waterfall process.

1.2 Scope

EUMS is a web application for a utility office that supplies electricity, water and piped gas. It manages customers, connections and meters, records meter readings, generates slab-based bills automatically, accepts payments, and handles complaints and outages. It provides a staff portal (administrator and employees) and a customer self-service portal. Real payment gateways, smart meters, SMS/e-mail and mobile apps are out of scope.

1.3 Definitions and abbreviations

Term	Meaning
Connection	A customer's supply of one utility at their address
Meter reading	The cumulative value shown on a meter on a date
Consumption	Current reading − previous reading
Slab tariff	Progressive pricing: each band of units has its own rate
Fixed charge	Monthly charge independent of usage
Outstanding	Bill total − amount already paid
kWh / kL / SCM	Units for electricity / water (kilolitre) / gas (standard cubic metre)
RBAC	Role-based access control
MoSCoW	Must / Should / Could / Won't prioritisation

1.4 References

IEEE Std 830-1998; course brief for the DBMS and SEPM projects; state electricity tariff orders (LMV-1/LMV-2) used as a model for slab structure.

1.5 Overview

Section 2 describes the product in general terms. Section 3 lists interfaces. Sections 4–6 give the functional, non-functional and data requirements.

2. Overall Description

2.1 Product perspective

EUMS is a new, self-contained system that replaces paper registers and spreadsheets. It is a three-tier web application (browser, Flask server, SQLite database).

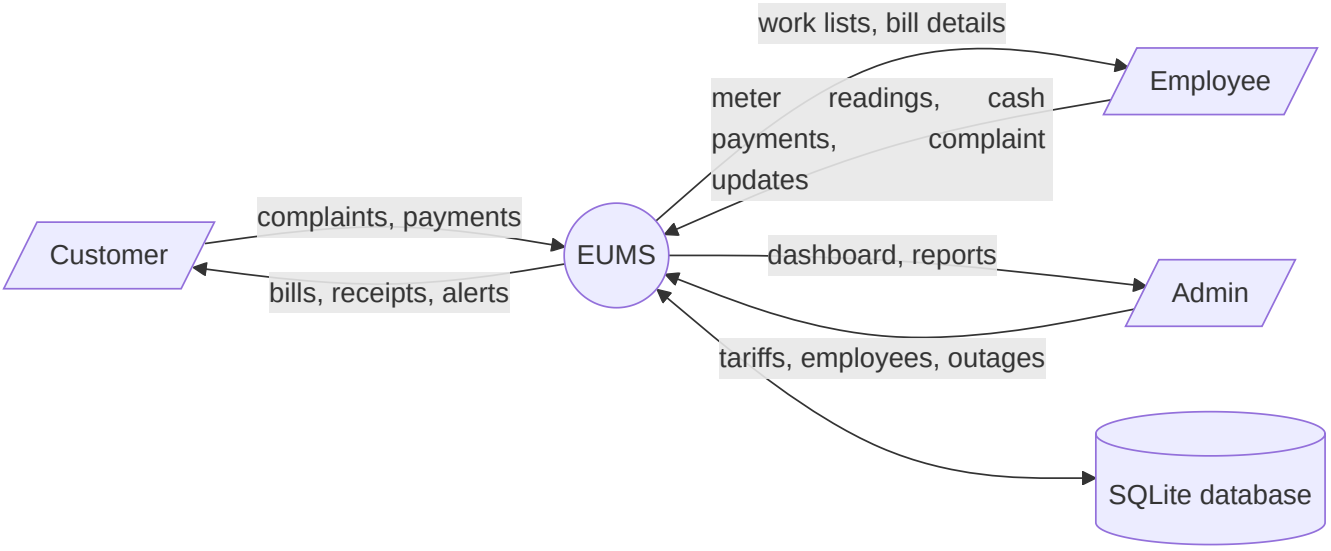


Figure 1: Context diagram

2.2 Product functions

- Manage users, customers, employees, connections and meters.
- Record meter readings with validation; generate itemised slab-based bills.
- Accept online and counter payments, including partial payments, and issue receipts.
- Manage complaints and outages; notify customers.
- Maintain tariffs; show dashboards and reports.

2.3 User classes and characteristics

User class	Characteristics	Access
Administrator	Office manager; moderate computer skills	Everything, including tariffs, employees, reports
Employee	Meter reader, billing officer or technician; basic skills; may use a phone in the field	Customers, connections, readings, billing, payments, complaints, outages
Customer	General public, residential or commercial; uses UPI apps	Only their own connections, bills, usage, complaints, outages in their area

2.4 Operating environment

Server: any PC with Python 3.10+ and Flask (Windows, Linux or macOS). Client: any modern browser (Chrome, Edge, Firefox) on desktop or phone. Database: SQLite 3 file.

2.5 Design and implementation constraints

- Free and open-source software only; must run on a student laptop without a database server.
- Relational database with constraints, triggers and views (DBMS course requirement).
- No passwords, API keys or real personal data in the source code or sample data.

2.6 Assumptions and dependencies

- One connection per utility per customer; one active meter per connection; meters are cumulative.
- Tariff depends only on the utility and the customer category (residential / commercial).
- Bills are due 15 days after the bill date. Payment gateway is simulated.

3. External Interface Requirements

3.1 User interfaces

Web pages with a left navigation menu by role, a top bar with the user's name, cards for summaries, tables for lists and simple forms. Errors are shown as red messages at the top of the page; successes in green. Layout adapts to screens narrower than 800 px.

3.2 Software interfaces

Python Flask 3 web framework; SQLite 3 via Python's `sqlite3` module; Chart.js 4 in the browser. A JSON endpoint `GET /staff/api/meters/<id>` returns the previous reading for the reading form.

3.3 Communication interfaces

HTTP between browser and server (HTTPS in a production deployment). Session cookie signed with a secret key.

4. Functional Requirements

ID	Module	Requirement	Priority	Status
FR-01	Authentication	Users log in with username/email and password; passwords are stored as salted hashes; users can log out.	Must	Implemented
FR-02	Authorisation	Each page is restricted to its roles (Admin, Employee, Customer). Unauthorised access returns HTTP 403; not logged in redirects to login.	Must	Implemented
FR-03	Customers	Staff register a customer (name, email, 10-digit phone, address, area, category, initial password). Email must be unique. A customer code (C1001…) is generated.	Must	Implemented
FR-04	Customers	Staff search customers by name, code, area or phone and open a customer profile showing connections and recent bills.	Should	Implemented

FR-05	Connections	Staff create a connection for a customer and utility; the tariff is chosen from the customer category; a meter with a starting reading is created. Only one connection per utility per customer.	Must	Implemented
FR-06	Meter readings	Employee selects a meter; the system shows customer, utility and previous reading, and calculates consumption as the current value is typed.	Must	Implemented
FR-07	Meter readings	A reading lower than the previous reading, or dated on/before the last reading, is rejected.	Must	Implemented
FR-08	Billing	Saving a reading generates a bill automatically: slab charges + fixed charge + tax, with itemised rows, bill number and due date (+15 days).	Must	Implemented
FR-09	Billing	Staff list bills filtered by status and search; any user can open an allowed bill and print / save it as PDF.	Must	Implemented
FR-10	Payments	Customer pays a bill online by card, UPI, net banking or wallet (simulated); partial payments allowed; a receipt number is generated.	Must	Implemented
FR-11	Payments	Payment larger than the outstanding amount is rejected. Bill status changes to partial or paid automatically.	Must	Implemented
FR-12	Payments	Staff record counter (cash) payments and view all payments with totals by method.	Should	Implemented
FR-13	Complaints	Customer raises a complaint for a connection with urgency; staff view by status, assign to an employee, change priority/status and add resolution notes.	Must	Implemented
FR-14	Outages	Staff report an outage (utility, area, reason, start, estimated end); future outages are "scheduled". Staff mark outages resolved.	Should	Implemented
FR-15	Notifications	Customers are notified in-app of new bills, payments, outages in their area for their utilities, and complaint status changes.	Should	Implemented
FR-16	Tariffs	Admin edits slab rates, fixed charge and tax per plan; changes apply only to new bills.	Must	Implemented
FR-17	Employees	Admin views employees with workload and adds new employees.	Should	Implemented
FR-18	Dashboards	Admin/employee dashboard shows KPIs, billed amount per utility (6 months), collections, recent activity. Customer dashboard shows connections, current bills, monthly bill chart and recent bills.	Should	Implemented
FR-19	Reports	Admin sees usage per utility, area-wise consumption, billed amount by category, collection efficiency and overdue bills.	Could	Partial

FR-20	Customer portal	Customer views connections, usage history (per-utility charts), outages in their area and edits phone/address and password.	Should	Implemented
FR-21	Billing	Late-payment surcharge and automatic overdue reminders.	Could	Planned
FR-22	Billing	Downloadable server-generated PDF invoice and e-mail delivery.	Could	Planned
FR-23	Meters	Meter replacement workflow (mark faulty/replaced, new meter with baseline reading).	Should	Planned
FR-24	Data export	Export reports to CSV/Excel.	Could	Planned
FR-25	Payments	Real payment-gateway integration.	Won't (now)	Planned

Table 1: Functional requirements with MoSCoW priority and status

Detailed use-case descriptions and user stories with acceptance criteria are in `02_Requirements/Use_Cases_or_User_Stories.pdf`.

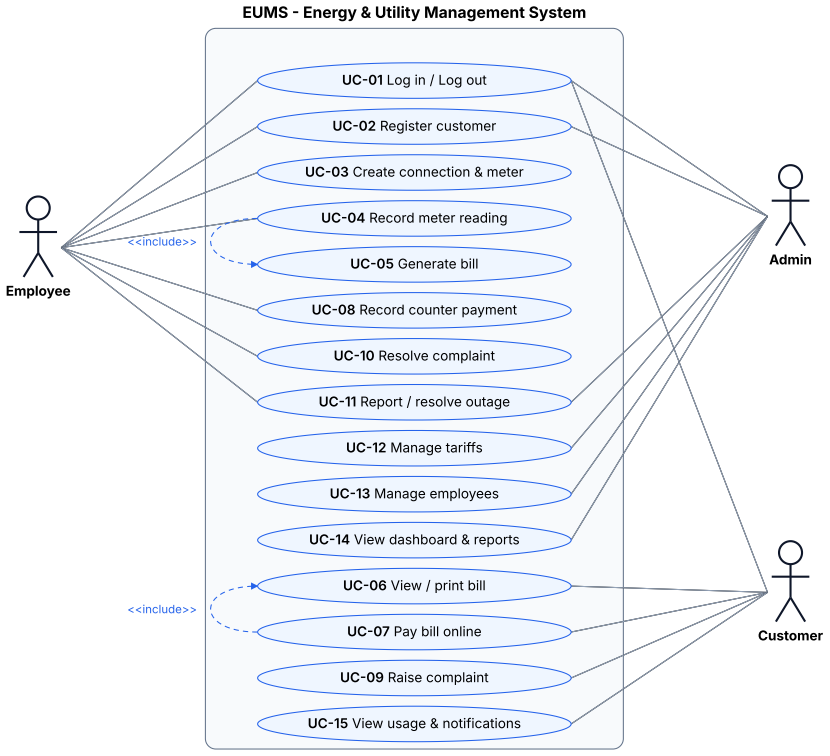


Figure 2: Use-case diagram

5. Non-Functional Requirements

ID	Category	Requirement	How it is met / measured
NFR-01	Security	Passwords never stored in plain text.	Werkzeug salted hash (scrypt/PBKDF2)

NFR-02	Security / privacy	A customer can access only their own records.	Every portal query filtered by logged-in customer; 403 otherwise (test IT-05)
NFR-03	Security	No secrets in source code.	`config.py` is git-ignored; only `config.example.py` is committed
NFR-04	Data integrity	Business rules hold even if application code has a bug.	CHECK, UNIQUE, FK constraints and 7 triggers in the database
NFR-05	Accuracy	Bills are correct to the paisa.	Amounts rounded to 2 decimals; unit tests UT-01..06
NFR-06	Auditability	Issued bills never change when tariffs change.	Charges stored on the bill (snapshot); test IT-14
NFR-07	Performance	Pages load in under 1 second for 1,000 customers on a laptop.	Indexes on readings, bills, customers, complaints, notifications
NFR-08	Usability	A new employee can enter a reading without training.	One task per page, live consumption, plain-language errors
NFR-09	Portability	Runs on Windows, Linux, macOS with Python 3.10+.	Only Flask needed; SQLite is built into Python
NFR-10	Responsiveness	Usable on a phone screen (meter readers in the field).	Responsive CSS layout below 800 px
NFR-11	Maintainability	Code is easy for students to read and change.	Small files by role; billing logic separate from web code; automated tests
NFR-12	Reliability	A failed operation leaves no half-saved data.	Each action in one transaction; rollback on error

Table 2: Non-functional requirements

6. Data Requirements

The system stores the following entities: users, customers, employees, utilities, tariff plans and slabs, connections, meters, meter readings, bills and bill items, payments, complaints, outages, notifications and an activity log. Key business rules on data:

- BR-1: A meter reading must be ≥ the previous reading of the same meter, and later in date.
- BR-2: Exactly one bill per meter reading; bill charges are fixed once issued.
- BR-3: Total payments on a bill can never exceed the bill total. Status is unpaid, partial or paid.
- BR-4: Customer email, customer code, connection number, meter number, bill number and receipt number are unique.
- BR-5: A customer has at most one connection per utility.

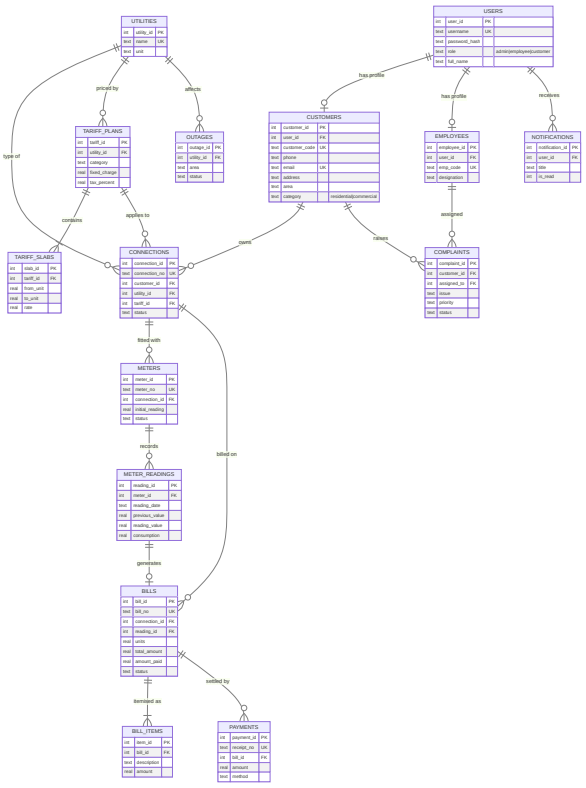


Figure 3: Logical data model (ER diagram)